# Procura

## A task manager that schedules work the way an operating system does

Six CPU-scheduling policies, simulated resource contention and deadlocks, and automatic detection, diagnosis and recovery.

Harshit Chhabi | Operating Systems | github.com/harshitchhabi/procura

# What this review covers

| Part | Slides | What you will see |
|---|---|---|
| 1 Problem and idea | 3 to 6 | Why a to-do list needs an operating system's methods, and how a task maps onto a process |
| 2 How it works | 7 to 21 | Every module and function: process model, schedulers, resources, detection, diagnosis, recovery, API, interfaces |
| 3 Live demo | 22 to 31 | Create tasks, compare schedulers, memory contention, a deadlock built and resolved with AI diagnosis, the terminal dashboard |
| 4 Evidence | 32 to 38 | 495-workload study, two fairness measures, evaluation of the AI answers and the search, bugs found |
| 5 Limits and questions | 39 to 43 | What is not claimed, future work, likely questions, commands to run each part alone |

# Task managers ignore forty years of scheduling research

## Priority is a label

High, Medium or Low is whatever the user typed. Nothing combines the deadline, the time already waited and the cost of the task into an order.

**OS answer: scheduling policies with aging**

## Fairness is never measured

No tool shows whether attention is spread evenly, or whether one task has been quietly starved.

**OS answer: Jain's fairness index and slowdown**

## Stuck tasks are not handled

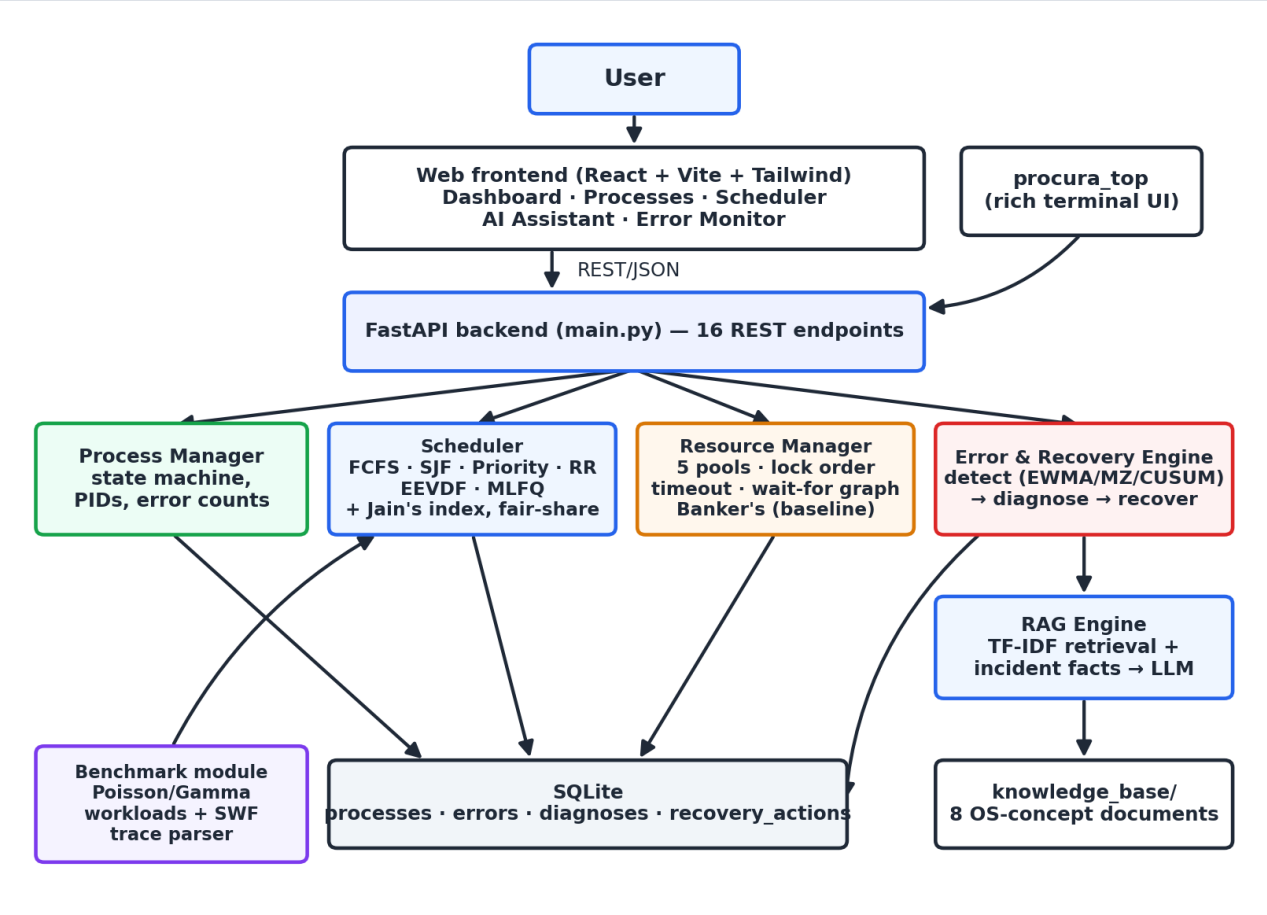
A task blocked on an exhausted resource shows a flag. Nothing explains why, and nothing repairs it.

**OS answer: detect, diagnose, recover**

# The idea: map each part of a task onto an OS concept

| In a to-do list | In Procura (OS term) | What it makes possible |
|---|---|---|
| A task | A process with a PID | A strict lifecycle: NEW, READY, RUNNING, BLOCKED, COMPLETED, TERMINATED |
| Estimated effort | Burst time, in minutes | Shortest-job-first, Gantt charts, waiting-time arithmetic |
| Importance and deadline | Effective priority score | An order that also rewards waiting, so nothing starves |
| Time already waiting | Aging | Guaranteed progress for low-priority tasks |
| Memory, files, connections a task needs | Requests against five fixed pools | Realistic contention: a request can fail |
| Waiting on someone else's task | A wait-for edge | Deadlock detection when the waits form a cycle |
| A task that stalls | A detected failure | Explanation by the AI assistant, repair by fixed rules |

# One backend, four subsystems, two ways in




- **Interfaces.** React web app with five screens, and procura_top, a terminal dashboard. Same REST API.
- **main.py.** FastAPI, 16 endpoints, glues the subsystems together.
- **os_sim/.** Process manager, scheduler, resource manager.
- **recovery/.** Detector, diagnosis, recovery scripts.
- **rag/.** Tokenizer, TF-IDF search, assistant.
- **benchmark/.** Workload generator, SWF trace parser, metrics, statistical study.
- **SQLite.** One file, raw sqlite3, no ORM.

# Follow one task through the whole system

1

## Form

Processes.jsx calls api.js, which sends POST /processes.

2

## Admission

main.py validates; process_manager.create_process assigns a PID and saves a row; state NEW becomes READY.

3

## Scheduling

GET /schedule runs run_schedule() in scheduler.py and returns order, Gantt slices and metrics.

4

## Resources

ResourceManager.allocate() checks the pool; if it does not fit, the request fails and a wait-for edge is recorded.

5

## Detection

POST /detect runs run_detection() over all processes and the resource state.

6

## Diagnosis

diagnosis.diagnose() asks the RAG assistant to explain the finding, with the recorded facts as ground truth.

7

## Recovery

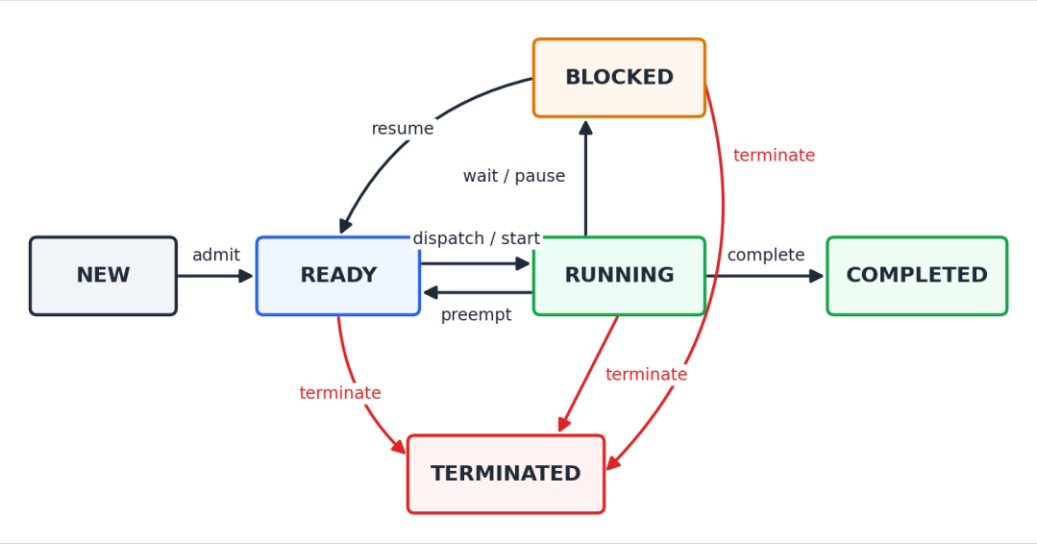
recovery.run_recovery() applies a fixed rule: retry, suspend a lower-priority holder, or terminate.

8

## Record

Error, diagnosis and action are written to SQLite and shown in the Error Monitor.

# Process model: a PID and an enforced lifecycle




| From | May move to |
|---|---|
| NEW | READY |
| READY | RUNNING, TERMINATED |
| RUNNING | READY, BLOCKED, COMPLETED, TERMINATED |
| BLOCKED | READY, TERMINATED |
| COMPLETED, TERMINATED | none |

| Function (process_manager.py) | Does |
|---|---|
| `create_process` | Assigns the next PID, saves the row, moves NEW to READY |
| `get_process / list_processes` | Read one process, or all (optionally by state) |
| `transition_state` | Checks the move with validate_transition, then saves it |
| `start_process` | READY to RUNNING, records started_at |
| `pause_process / resume_process` | RUNNING to BLOCKED / BLOCKED to READY |
| `complete_process / terminate_process` | Move to a final state |
| `increment_error_count` | Counts failures against a process |
| `delete_process` | Removes the row |
| `validate_transition (states.py)` | Raises InvalidTransitionError for an illegal move |

# Persistence: one SQLite file with four tables

| Table | Holds |
|---|---|
| `processes` | PID, name, description, priority, deadline, burst time, arrival time, memory and CPU requirement, state, error count, timestamps |
| `errors` | One row per detected failure: process, type, raw details, time |
| `diagnoses` | The explanation written for each error |
| `recovery_actions` | Action type and target PID for each error |

| Function (database.py) | Does |
|---|---|
| `init_db / reset_db` | Create the tables; drop and recreate them (seed script, tests) |
| `create_process / get_process / list_processes / update_process / delete_process` | CRUD for the processes table |
| `log_error / log_diagnosis / log_recovery_action` | Append one record to each history table |
| `get_error_log` | Join errors, diagnoses and recovery_actions into one history |
| `get_connection / db_cursor / now_iso` | Connection helpers and the timestamp format |

# Effective priority: five terms set the order

$$E = U + D + W + R + S$$

| Term | Function | Rule | Range |
|---|---|---|---|
| User priority U | `user_priority_score` | High 90, Medium 50, Low 20; a number is clamped to 0 to 100 | 20 to 90 |
| Deadline urgency D | `deadline_urgency_score` | 0 beyond 48 h; 100 once passed; 100 x (1 - h/48) between | 0 to 100 |
| Aging W | `waiting_time_score` | 2 points per minute in READY, capped at 100 | 0 to 100 |
| Resource cost R | `resource_requirement_score` | up to 60 for memory (of 2048 MB) plus up to 40 for CPU | 0 to 100 |
| Fair share S | `fair_share_score` | Penalty for using more than 1/n of decayed past use; half-life 60 min (_decay_factor) | -100 to 0 |
| Total | `effective_priority_score` | Sum of the five; used by priority_scheduling and by the recovery rules | |

Live check: a Medium task outranked a High one (225.9 against 218.0) because both deadlines had passed and the Medium task needed more resources. The formula is working as written; whether the weights suit a user is a design choice.

# FCFS, SJF and Priority run one task at a time to completion

| Function (scheduler.py) | Does |
|---|---|
| `fcfs` | Dispatch in arrival order |
| `sjf` | Among arrived jobs, run the shortest burst |
| `priority_scheduling` | Among arrived jobs, run the highest effective priority |
| `_arrival_aware_order` | The shared rule: pick the best arrived job; idle only if nothing has arrived |
| `_run_nonpreemptive` | Turns an order into Gantt slices and metrics |

**Hand-worked example (also the unit test)**

| Job | Arrives | Burst |
|---|---|---|
| P1 | 0 | 5 |
| P2 | 1 | 3 |
| P3 | 2 | 8 |
| P4 | 3 | 6 |

| Policy | Order | Avg wait |
|---|---|---|
| FCFS | 1, 2, 3, 4 | 5.75 |
| SJF | 1, 2, 4, 3 | 5.25 |
| Round Robin q=4 | interleaved | 9.25 |

A bug found by running it: the first SJF pre-sorted all jobs and idled the CPU waiting for late arrivals. That inverted the result (SJF looked worse than FCFS). The original test encoded the bug.

# Round Robin and EEVDF share the CPU in time slices

## Round Robin

- FIFO ready queue; each turn lasts at most the quantum (4 minutes by default).
- A preempted task goes behind anything that arrived during its slice.
- Wait between turns is at most (n - 1) x quantum.
- **round_robin(processes, quantum)**

## EEVDF (Linux 6.6 and later)

```
weight   w  = 1024 / 1.25^nice      High -5, Medium 0, Low +5
deadline d  = v + slice * 1024 / w
after a slice:  v <- v + run * 1024 / w
eligible  when  lag = (weighted average v) - v  >= 0
dispatch  the eligible task with the earliest d  (min-heap)
```

- Heavier tasks have a slower virtual clock, so they become eligible, and run, more often.
- A starved task's lag grows until it is eligible: no starvation.
- **nice_from_priority, weight_from_priority, eevdf(processes, base_slice)**

A simulation of the published rules, not a port of the kernel. Per-task requested slices are not modelled.

# MLFQ lets the system learn which tasks are short

```
Queue 0   quantum 4   new tasks enter here
Queue 1   quantum 8   after using the Queue 0 allotment
Queue 2   quantum 16  after using the Queue 1 allotment

Every 50 minutes: boost everything back to Queue 0
```

- Rule 1: a higher queue always runs before a lower one.
- Rule 2: tasks in the same queue take turns (round robin).
- Rule 3: a new task starts in the top queue.
- Rule 4: using the whole allotment at a level moves the task down. The allotment is cumulative, so yielding early does not reset it.
- Rule 5: a periodic boost returns every task to the top, which bounds starvation.

| Function (scheduler.py) | Does |
|---|---|
| `mlfq(processes, num_queues, base_quantum, boost_interval)` | Runs the five rules; returns order, Gantt, metrics |
| `admit / maybe_boost` | Inner helpers: bring arrivals in; apply the boost |
| `run_schedule(algorithm, ...)` | Dispatch to any of the six policies by name |

> Seven-task demo: average response time FCFS 431.4 min, SJF 173.5, Round Robin and MLFQ 11.9, EEVDF 17.1.

# Every schedule is scored with the same metrics

```
turnaround = completion - arrival
waiting    = turnaround - burst
response   = first dispatch - arrival
utilisation = busy time / makespan
Jain  J(x) = (sum x)^2 / (n * sum x^2)    range 1/n to 1
slowdown = max(turnaround / max(burst, 1 minute), 1)
delay fairness = Jain's index over 1 / slowdown
```

| Function | Does |
|---|---|
| `_build_metrics` | Per-process and average metrics for any schedule |
| `jains_fairness_index` | The formula above |
| `bounded_slowdown` | Slowdown with the one-minute bound |
| `_parse_arrival, _relative_arrivals` | Accept ISO dates or numbers; shift to the earliest arrival |
| `benchmark/metrics.py summarize` | Adds makespan, starvation count, max wait for the benchmark |

Jain's index over CPU received is identical for all six policies, because every policy gives each task exactly its burst time. That is why the delay-based index was added: it does separate them (0.35 to 0.69 on the seeded run).

# Resource manager: five finite pools and a wait-for graph

| Pool | Capacity | Unit |
|---|---|---|
| cpu | 100 | percent |
| memory | 2048 | MB |
| file | 64 | handles |
| database | 20 | connections |
| network | 1000 | Mbps |

> Live allocations are held in memory and reset when the server restarts. That is by design: it is a simulation, not a ledger.

| Function (ResourceManager) | Does |
|---|---|
| `allocate(pid, resource, amount)` | Grant, or fail and record a wait-for edge |
| `release / release_all_for_pid` | Free resources and clear edges |
| `available / utilization` | Free amount; use of all five pools |
| `wait_graph` | The wait-for graph as {pid: [pids it waits for]} |
| `detect_cycle` | Depth-first search; returns the PIDs in a cycle |
| `check_lock_order_violation` | Logs out-of-order requests (cpu, memory, file, database, network) |
| `check_timeouts` | Aborts waits longer than 30 seconds |
| `register_max_need, _would_be_safe` | Banker's-style avoidance (off by default) |
| `describe_amount` | '64 file handles', '1200 MB of memory' |

# Deadlock: four conditions, four defences

| Defence | How it works | Real-world model | Default |
|---|---|---|---|
| Lock-order check | Resources are taken in the order cpu, memory, file, database, network; out-of-order requests are logged | Linux lockdep | on |
| Timeout | A wait longer than 30 seconds is aborted | MySQL lock wait timeout | on |
| Cycle detection | Depth-first search of the wait-for graph; a cycle means deadlock; recovery terminates the lowest-priority task in it | PostgreSQL deadlock detection | on |
| Banker's algorithm | Grant a request only if some task could still finish afterwards | Textbook only | off |

| Coffman condition | In Procura's demo |
|---|---|
| Mutual exclusion | A file handle or connection has one holder at a time |
| Hold and wait | P1 holds the file handles while waiting for a connection |
| No preemption | Resources are not taken away from a holder by the system until recovery |
| Circular wait | P1 waits for P3 and P3 waits for P1 |

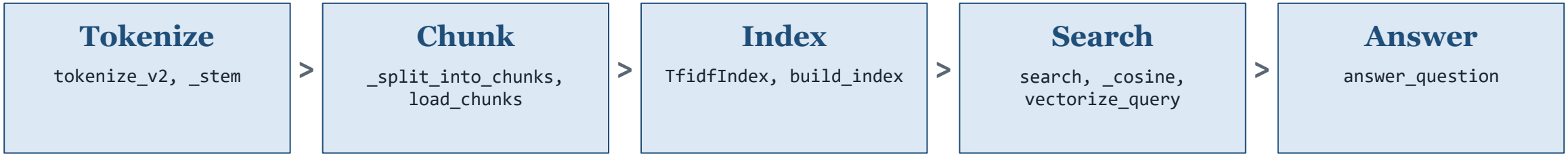
# Detection: three detectors run on every pass

| Function (detector.py) | Detects |
|---|---|
| `run_detection(processes, resource_mgr)` | Runs everything; skips COMPLETED and TERMINATED |
| `detect_memory_failure` | Unmet memory requirement larger than free memory |
| `detect_deadlock(resource_mgr, processes)` | A cycle in the wait-for graph; records who holds what, with names and priorities |
| `detect_burst_anomaly` | Elapsed/burst ratio out of line with the task's own history |
| `ewma_flag, modified_zscore_flag, cusum_flag` | The three NIST tests the anomaly detector can use |
| `reset_history, _record_ratio` | Rolling per-task history (30 values) |

```
EWMA     z = lambda*x + (1-lambda)*z      limits  z0 +/- k*sigma*sqrt(lambda/(2-lambda)),  lambda 0.25, k 3
Mod. Z   M = 0.6745 * (x - median) / MAD      flag when |M| > 3.5
CUSUM    S_hi = max(0, S_hi + x - mu - k)         flag when it exceeds h
```

> Limit: it flags abrupt departures from a task's own history, not a task that steadily runs over its estimate.

# Diagnosis starts with a search over eight documents




- **Tokenizer.** Keeps numbers and identifiers, indexes the parts of compounds, drops question filler, merges plurals. 106 stopwords.
- **Chunks.** 8 documents become 63 chunks of at most about 600 characters.
- **Index.** weight = term share in the chunk x idf; idf = ln((1 + N) / (1 + df)) + 1; 1,256 distinct terms.
- **Search.** Cosine similarity against every chunk; top 3; optional filter by document (for example Deadlocks, Error_Recovery).
- **Why TF-IDF.** No embedding model: installs in seconds, works offline, every step can be inspected.

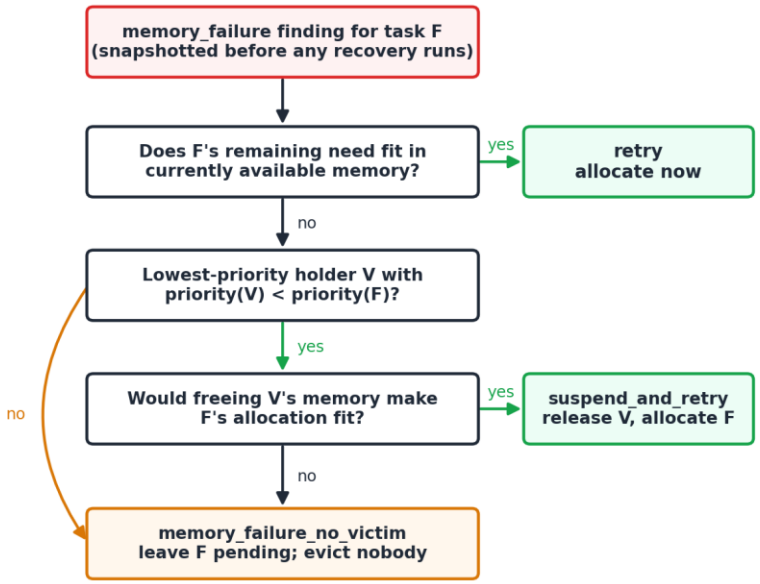
> The tokenizer is the lexer-like part of the project; the SWF trace parser is the parser-like part. Both can be run on their own (appendix).

# The assistant explains; fixed rules act

| Prompt part | Content |
|---|---|
| 1 Instruction | Treat the incident facts as ground truth; use the passages for concepts; state units; do not claim an action was the only one possible |
| 2 Process record | PID, name, priority, state, requirements (_format_process_context) |
| 3 Incident facts | Error details, action executed, rule applied, current pool state with units (_format_incident_facts) |
| 4 Passages | The top 3 chunks from the search |

| Function (assistant.py) | Does |
|---|---|
| `answer_question` | Search, build the prompt, call a provider, or fall back; returns answer, mode, sources, fallback_reason |
| `_build_prompt` | Assembles the four parts |
| `_call_groq / _call_anthropic / _call_openai` | One provider each, in that order of preference |
| `_post_json` | Retries rate limits (429) and server errors using the provider's own wait hint |
| `_extractive_fallback` | Offline answer: passages and facts, with the reason the model did not answer |

# Recovery: fixed rules, modelled on Kubernetes preemption



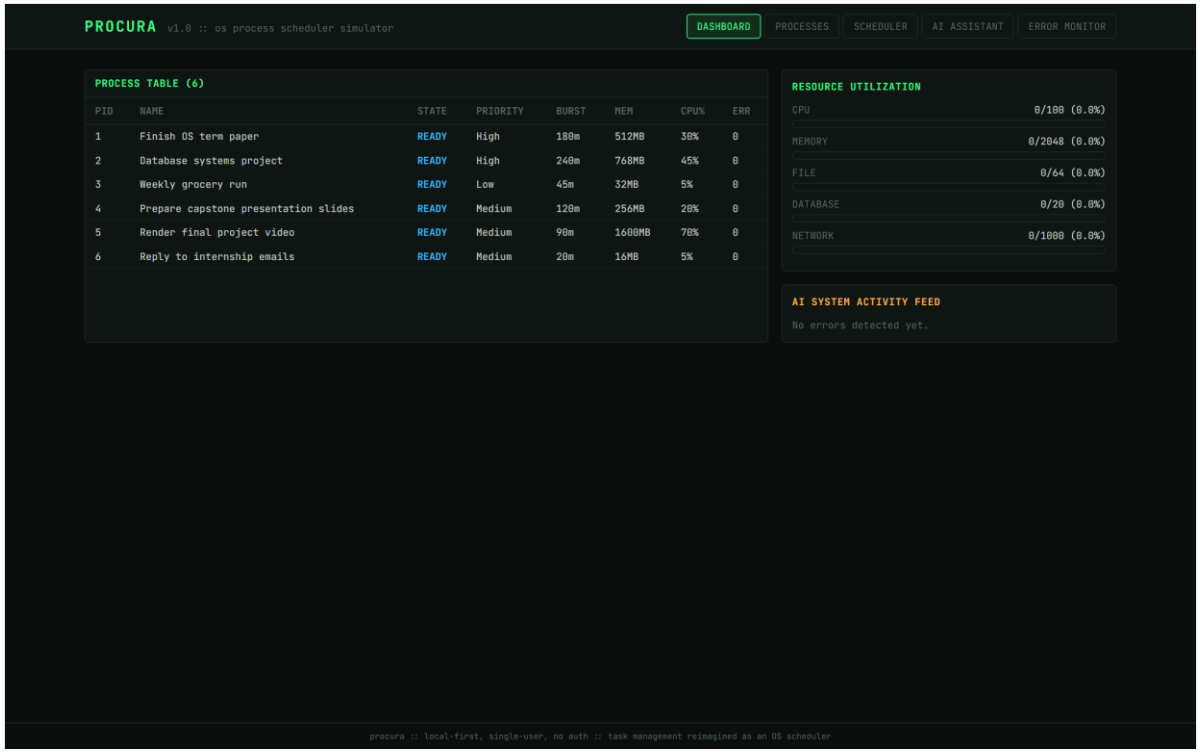

| Function (recovery.py) | Action |
|---|---|
| `recover_memory_failure` | retry, suspend_and_retry, or memory_failure_no_victim |
| `recover_deadlock` | terminate_and_resume: kill the lowest-priority cycle member, release its resources, unblock the rest |
| `recover_timeout / recover_crash` | stop_and_retry / restart: back to READY, error count +1 |
| `run_recovery` | Chooses the script by error type |
| `_lowest_priority_pid` | Finds the eviction candidate by effective priority |

> Never evict equal or higher priority work, and never evict unless it actually frees enough memory.

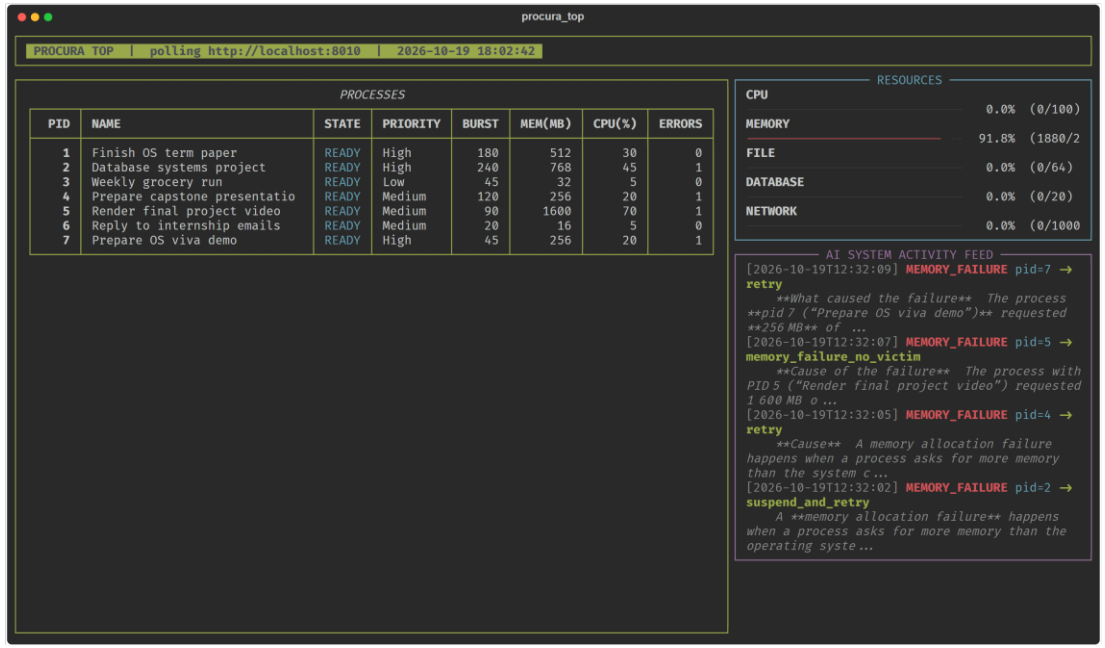
# The REST API: 16 endpoints in main.py

| Method | Endpoint | Purpose |
|---|---|---|
| GET | `/health` | Liveness check |
| GET, POST | `/processes` | List tasks; create a task |
| GET, PATCH, DELETE | `/processes/{pid}` | Read; change state (start, pause, resume, complete, terminate); delete |
| GET, POST | `/schedule` | Run one policy; returns order, Gantt data, metrics, fairness |
| GET | `/benchmark` | Run all six policies on a generated workload |
| GET | `/resources` | Use of each pool and the wait-for graph |
| POST | `/resources/allocate` | Allocate a resource to a process |
| POST | `/resources/release` | Release a resource |
| POST | `/resources/avoidance-mode` | Turn Banker's-style avoidance on or off |
| POST | `/detect` | Detect, diagnose and recover, once |
| POST | `/ask` | Ask the assistant, optionally about one process |
| GET | `/errors` | History of errors, diagnoses and recovery actions |

# Interfaces: five web screens and a terminal dashboard




*Web dashboard: process table, resource bars, AI activity feed.*




*procura_top: the same data in a terminal.*

Screens: Dashboard, Processes, Scheduler, AI Assistant, Error Monitor.

# Live demo: what will be shown, in order

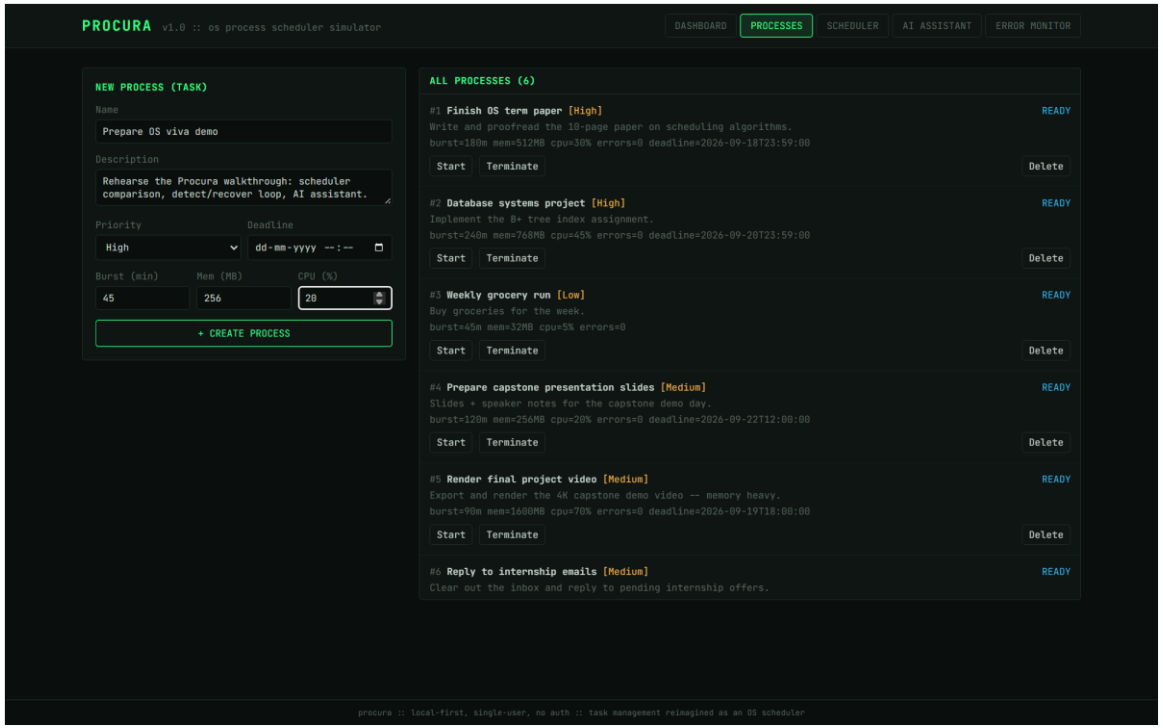
| # | Demo | Shows | Minutes |
|---|---|---|---|
| 1 | Create a task | Process model, PID, NEW to READY | 1 |
| 2 | Compare the six schedulers | Gantt charts, waiting time, Jain's index, the SJF trade-off | 3 |
| 3 | Memory contention and automatic recovery | Detection, retry, suspend-and-retry, no-victim | 2 |
| 4 | A deadlock, built and resolved with AI diagnosis | Wait-for graph, cycle, rule-based termination, grounded explanation | 4 |
| 5 | Ask the assistant | Retrieval plus recorded facts; offline fallback | 1 |
| 6 | Terminal dashboard and benchmark | procura_top, run_benchmark, the six-policy table | 1 |

```
cd backend
python seed_data.py
uvicorn main:app --port 8010

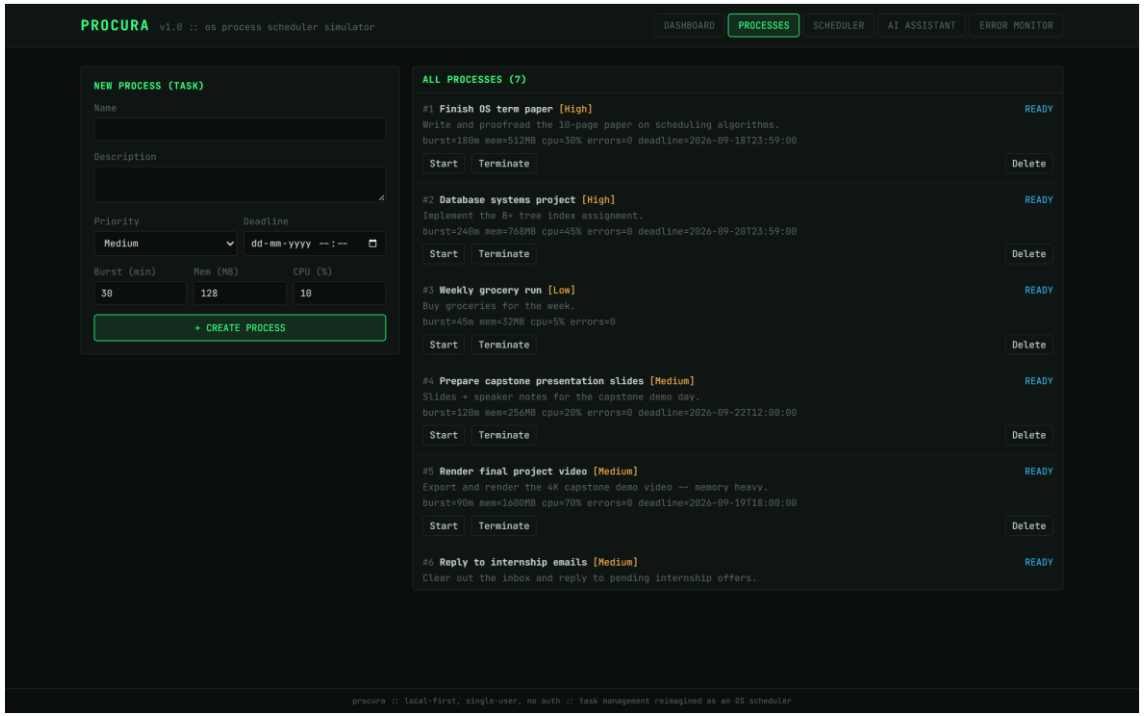
cd frontend
$env:VITE_API_URL="http://localhost:8010"
npm run dev            # then open http://localhost:5173
```

Port 8010 is used because another program occupies 8000 on this machine.

# Demo 1: create a task and watch it become a process



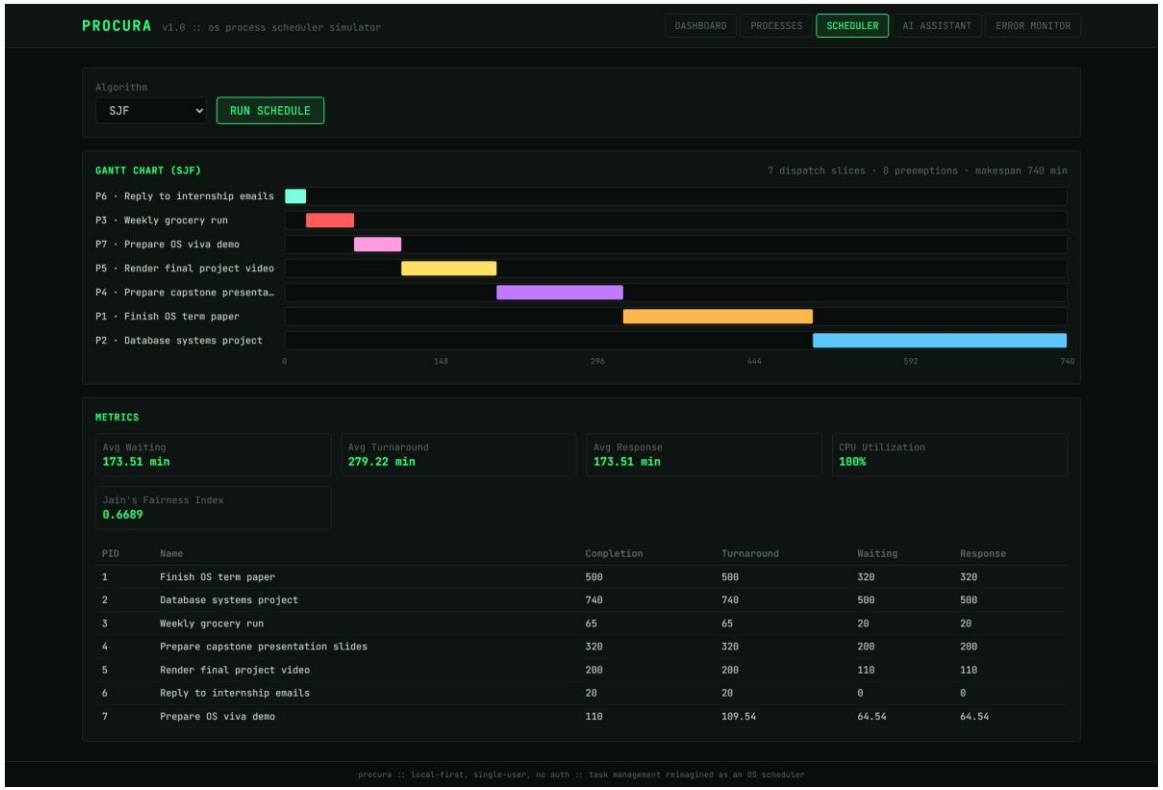

*1. Fill in the form: priority, burst time (minutes), memory (MB), CPU (%).*



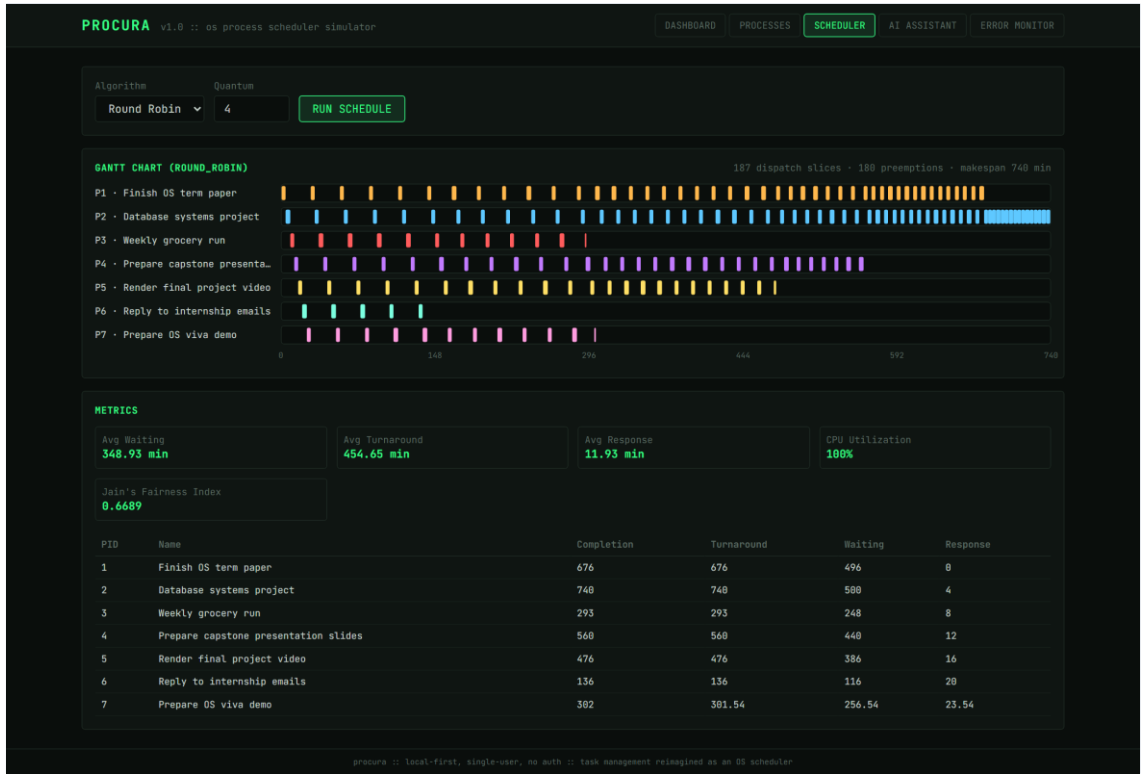

*2. The task is admitted as the next PID, state READY, saved in SQLite.*

`Functions: post_process, create_process, validate_transition, database.create_process`

# Demo 2: run the same tasks under different policies



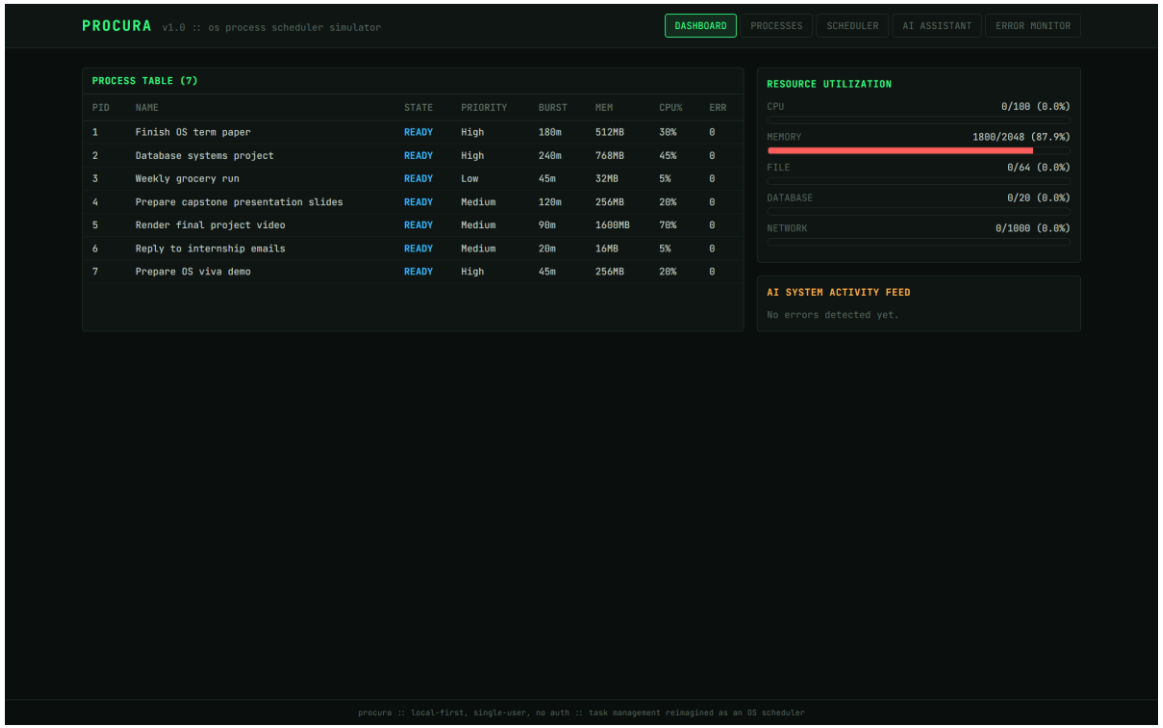

*SJF: shortest first. Lowest average wait, long tasks wait longest.*



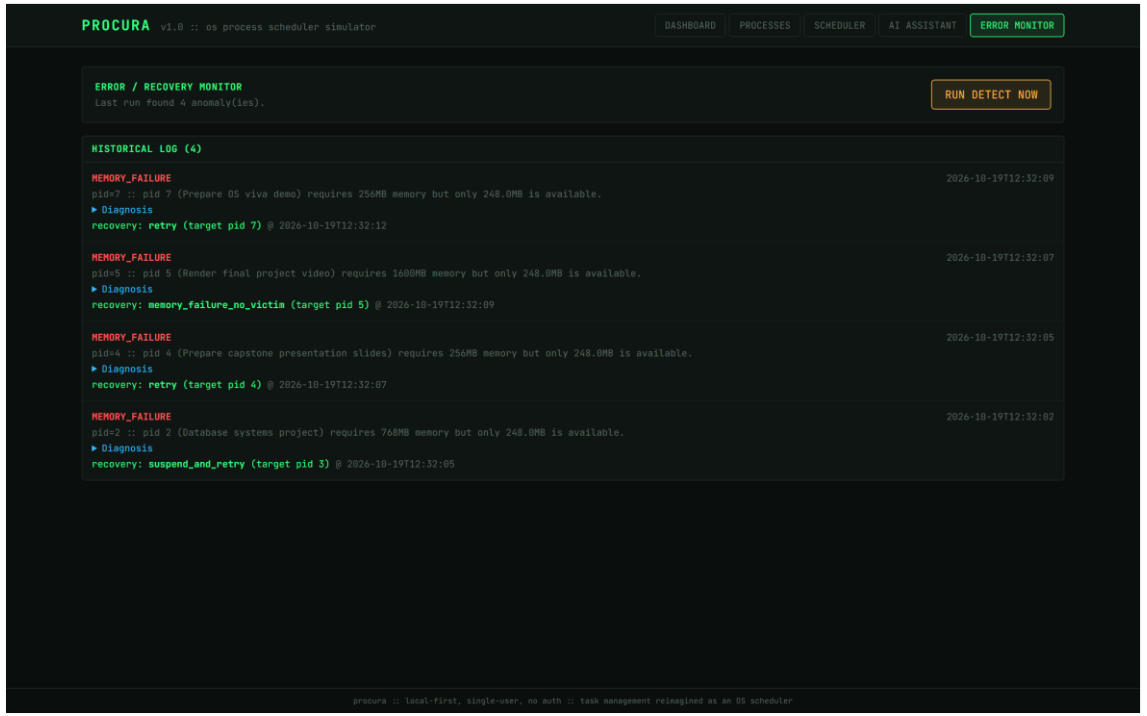

*Round Robin: every task gets a turn, so each starts early.*

`Functions: get_schedule, run_schedule, sjf, round_robin, _build_metrics`

# Demo 3: memory contention, detected and recovered




*Before: P3 holds 1200 MB and P1 600 MB; 248 MB of 2048 MB is free.*



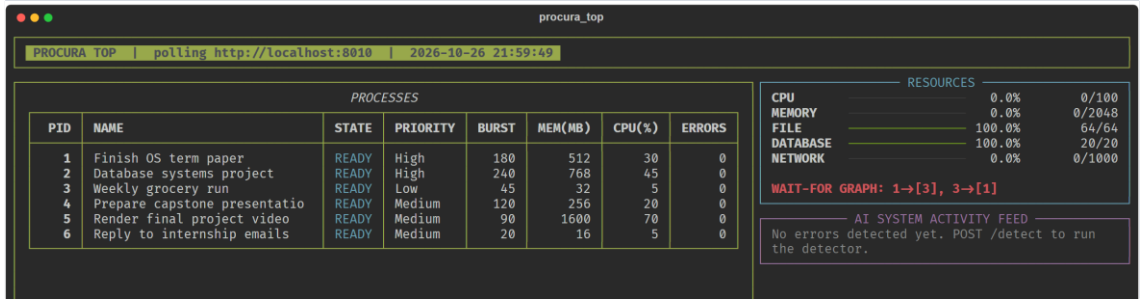

*After one detection pass: suspend_and_retry, retry, and memory_failure_no_victim.*

`Functions: detect_memory_failure, diagnose, recover_memory_failure, _lowest_priority_pid`

# Demo 4, step 1: build a deadlock with four API calls

```
A=http://localhost:8010; J="Content-Type: application/json"
curl -X POST $A/resources/allocate -H "$J" -d '{"pid":1,"resource":"file","amount":64}'
curl -X POST $A/resources/allocate -H "$J" -d '{"pid":3,"resource":"database","amount":20}'
curl -X POST $A/resources/allocate -H "$J" -d '{"pid":1,"resource":"database","amount":5}'    # fails
curl -X POST $A/resources/allocate -H "$J" -d '{"pid":3,"resource":"file","amount":5}'       # fails
curl $A/resources          # waiting_for_graph: {1:[3], 3:[1]}
```

| Call | Result |
|---|---|
| P1 takes 64 file handles | success |
| P3 takes 20 database connections | success |
| P1 asks for 5 connections | fails: P1 waits for P3 |
| P3 asks for 5 file handles | fails: P3 waits for P1 |

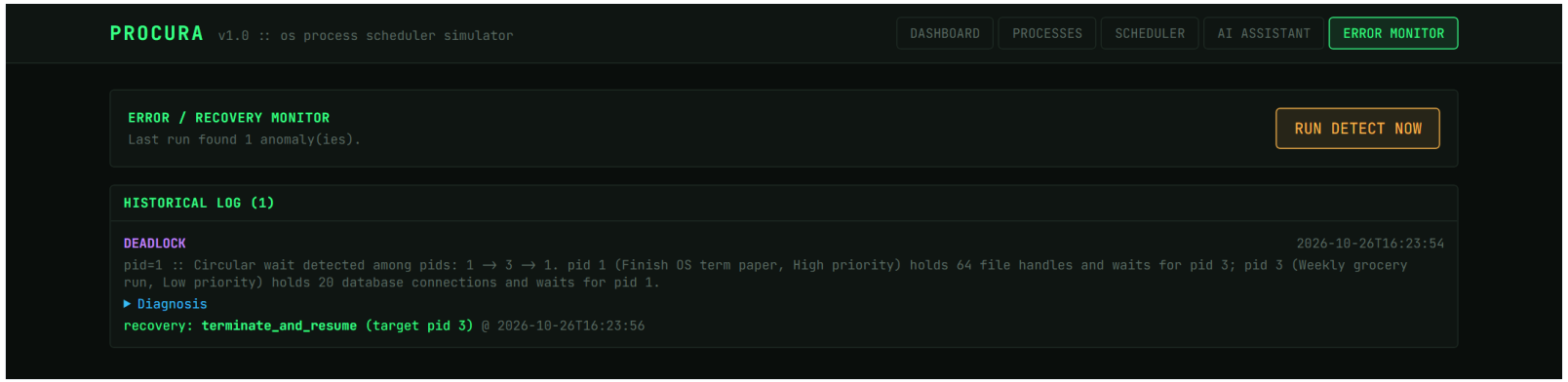



*procura_top while deadlocked: both pools full, WAIT-FOR GRAPH 1 to [3], 3 to [1].*

`Functions: allocate, _record_wait, wait_graph`

> Both pools are at 100% and each task waits for the other: hold and wait, plus circular wait.

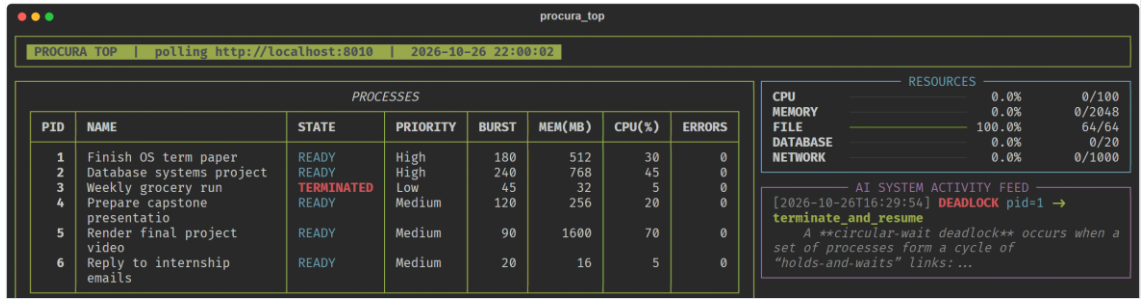
# Demo 4, step 2: one detection pass finds and resolves it




*Error Monitor after one detection pass: the cycle, each member's name and priority, and the action terminate_and_resume (target P3).*



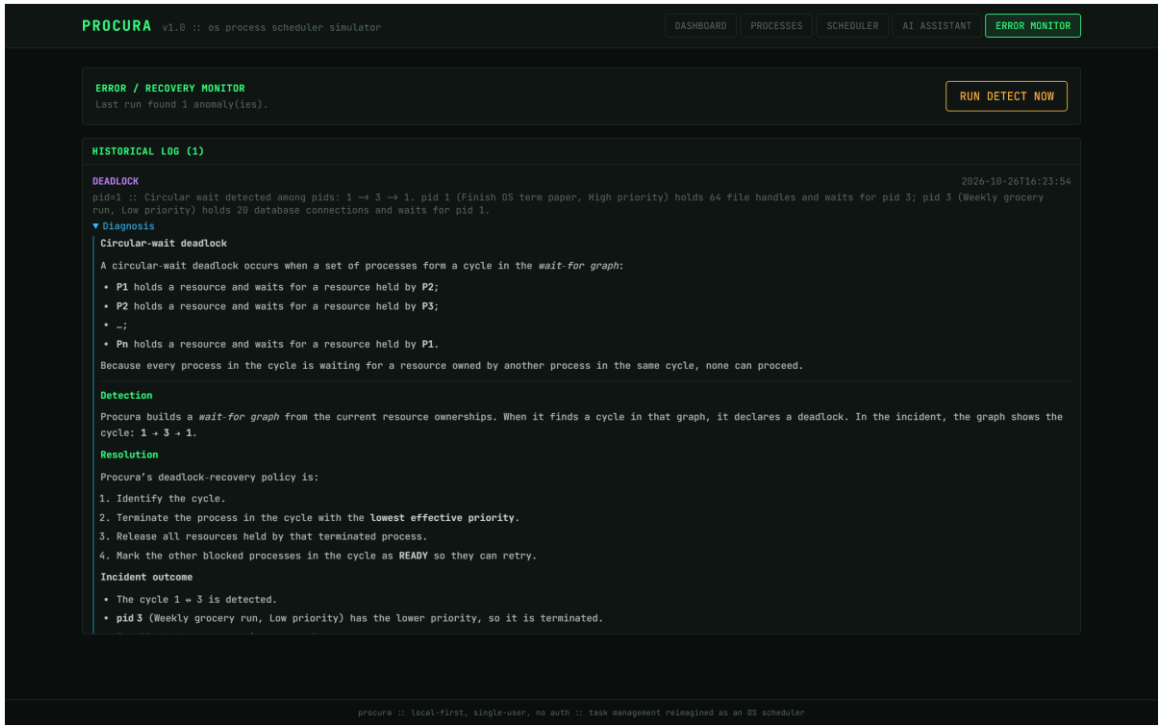

*Dashboard afterwards: P3 TERMINATED, connections released, P1 READY.*



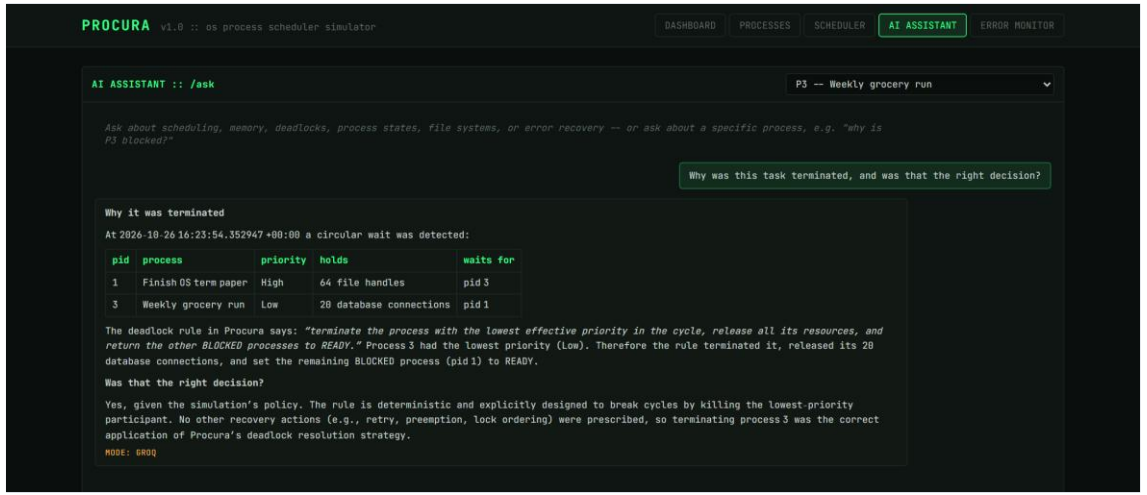

*procura_top afterwards: wait-for graph empty, event in the feed.*

`detect_deadlock > detect_cycle > diagnose > recover_deadlock > release_all_for_pid > log_recovery_action`

# Demo 4, step 3: the AI explains what happened and why




*Diagnosis written from the recorded cycle, priorities and the fixed rule.*




*Asking afterwards: why was P3 terminated, and was that right?*

`Functions: _incident_facts_for, answer_question, _build_prompt, _call_groq`

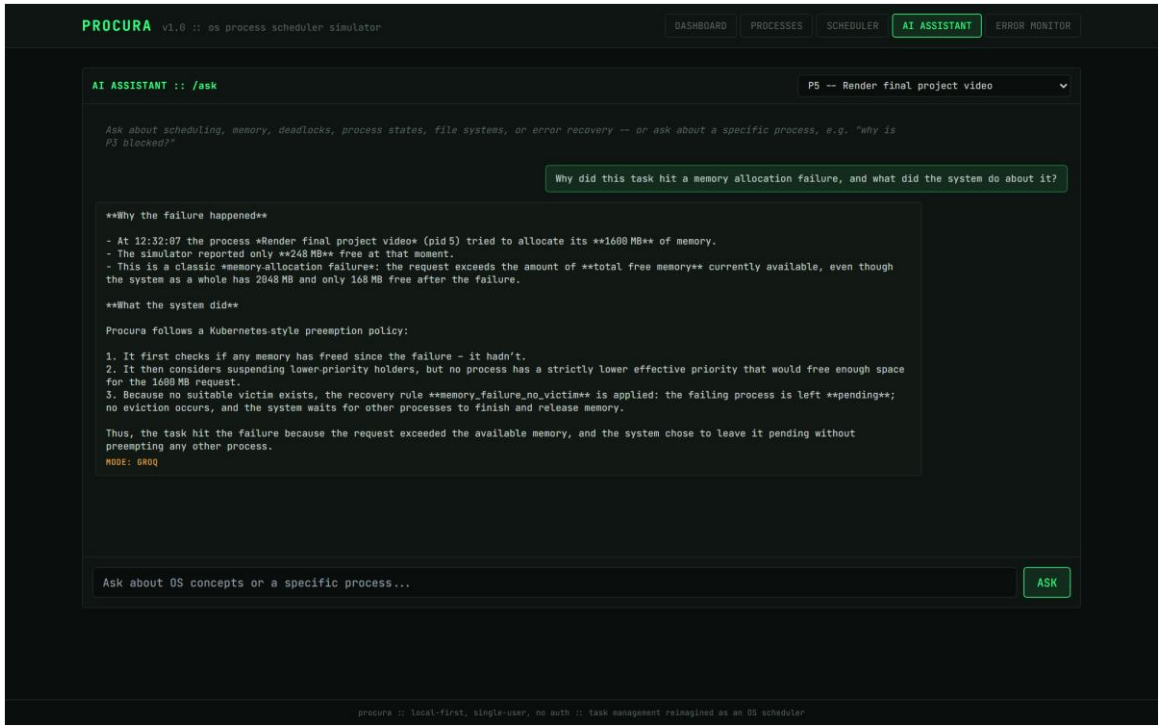
# Why the explanation can be trusted: what the model is given

| Given to the model | Example from this demo |
|---|---|
| The recorded incident | Circular wait among pids 1, 3, 1. pid 1 (Finish OS term paper, High priority) holds 64 file handles and waits for pid 3; pid 3 (Weekly grocery run, Low priority) holds 20 database connections and waits for pid 1. |
| The fixed policy | Terminate the process with the lowest effective priority in the cycle, release its resources, return the others to READY. |
| Passages from the knowledge base | Deadlocks and Error_Recovery documents (Coffman conditions, detection, runbooks). |

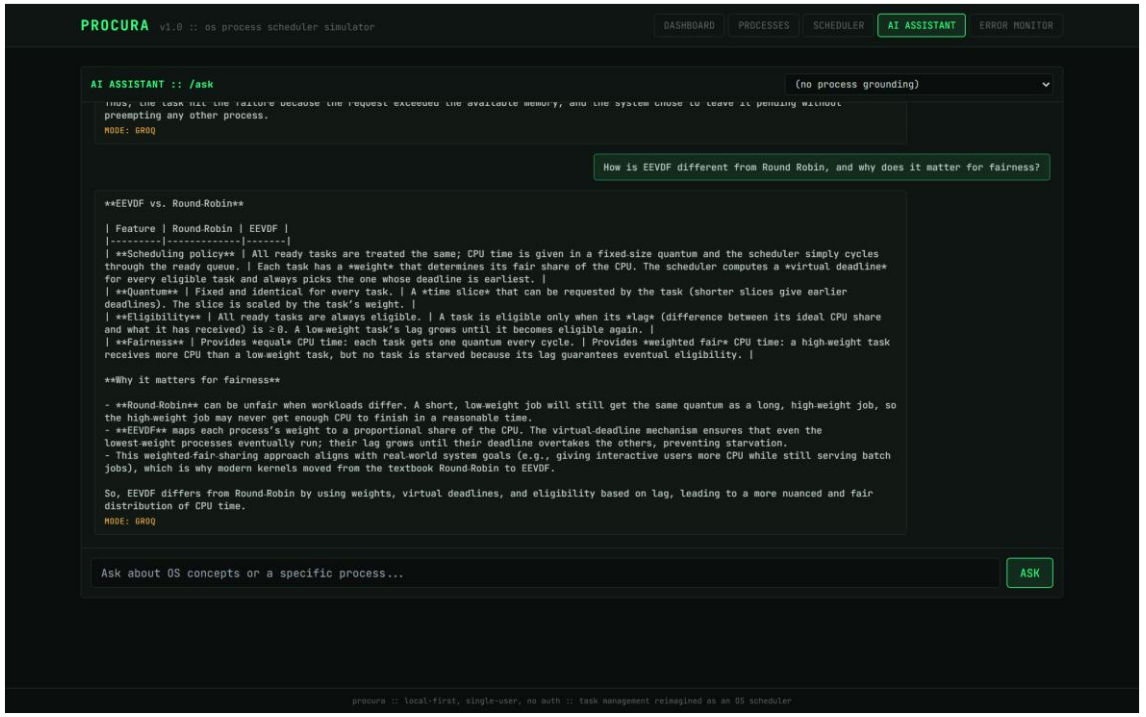
| Before the fix | After the fix |
|---|---|
| "pid 3's priority isn't specified ... if it is lower, Procura will terminate pid 3." | "pid 3 (Weekly grocery run, Low priority) has the lower priority, so it is terminated. Its 20 database connections are released." |

Found while capturing this demo. Fixed by recording names and priorities in the incident and passing the policy as a fact; two tests added.

# Demo 5: the assistant, grounded and general




*Selected process (P5): the answer uses the recorded facts.*



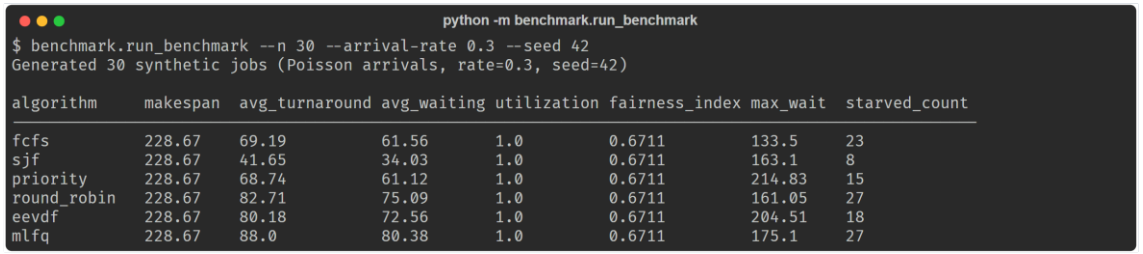

*No process selected: a concept question answered from the knowledge base.*

`Functions: ask, _incident_facts_for, search, answer_question`

# Demo 6: the terminal dashboard and the benchmark




*python -m cli.procura_top: process table, resource bars, wait-for graph, AI feed.*




*python -m benchmark.run_benchmark: all six policies on one workload.*

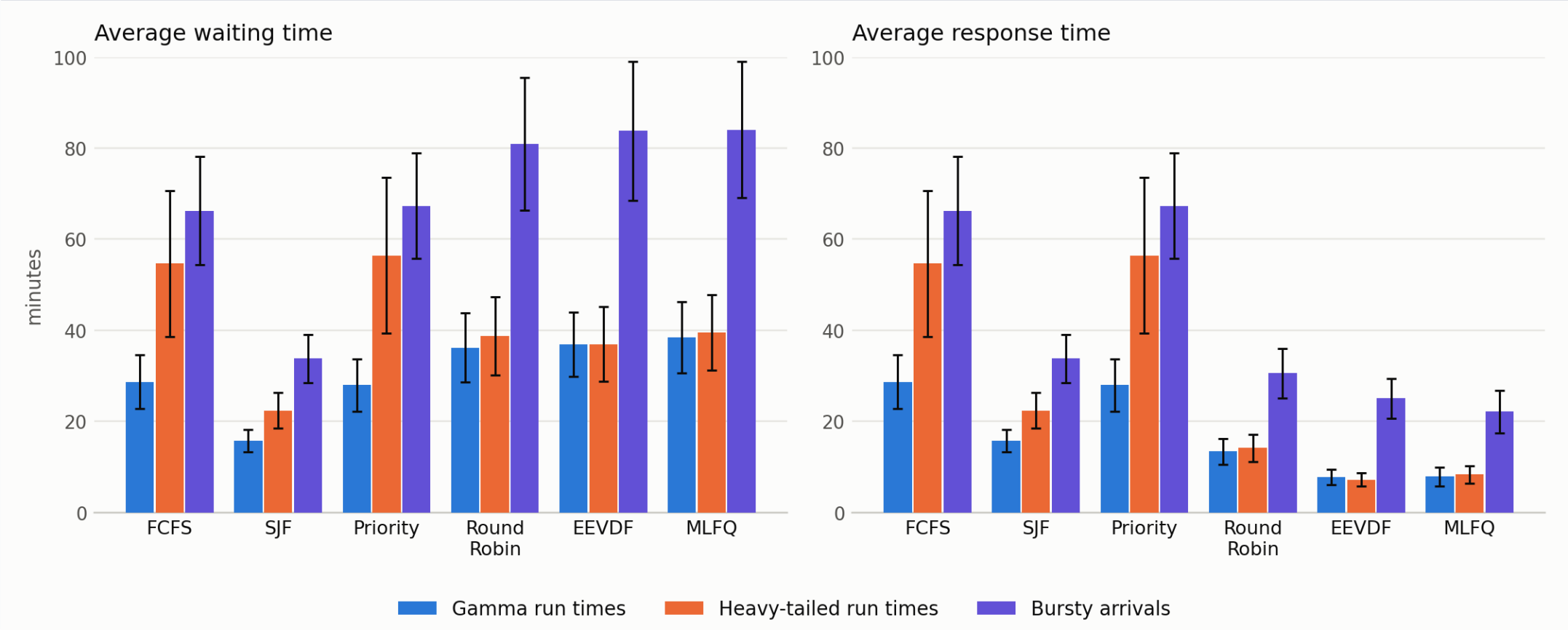
`Functions: build_process_table, build_resource_panel, build_ai_feed, render, run_all, print_table`

# One workload: SJF wins on average wait, loses on worst case

| Policy | Makespan | Avg wait | Max wait | Waited over 30 min | Jain (CPU) | Jain (delay) |
|---|---|---|---|---|---|---|
| FCFS | 228.67 | 61.56 | 133.50 | 23 | 0.6711 | 0.39 |
| **SJF** | 228.67 | **34.03** | **163.10** | **8** | 0.6711 | **0.69** |
| Priority | 228.67 | 61.12 | 214.83 | 15 | 0.6711 | 0.51 |
| Round Robin | 228.67 | 75.09 | 161.05 | 27 | 0.6711 | 0.38 |
| EEVDF | 228.67 | 72.56 | 204.51 | 18 | 0.6711 | 0.47 |
| MLFQ | 228.67 | 80.38 | 175.10 | 27 | 0.6711 | 0.35 |

Reproduce: python -m benchmark.run_benchmark --n 30 --arrival-rate 0.3 --seed 42. One run is one workload, so the next slides test whether it holds in general.

# The study: 495 workloads with confidence intervals




*Left: average waiting time. Right: average response time. 100 jobs, load 0.9, 50 workloads per shape, error bars are 95% confidence intervals.*

# Each policy against FCFS, across 9 settings

| vs FCFS (9 settings) | Average wait | Maximum wait | Average response | Delay fairness |
|---|---|---|---|---|
| **SJF** | **better in 9** | **worse in 9** | **better in 9** | **better in 9** |
| **Priority** | no difference | **worse in 9** | no difference | **better in 9** |
| **Round Robin** | **better 3, worse 6** | **worse in 9** | **better in 9** | **better 6, worse 0, n.s. 3** |
| **EEVDF** | **better 2, worse 6, n.s. 1** | **worse in 9** | **better in 9** | **better in 9** |
| **MLFQ** | **better 2, worse 6, n.s. 1** | **worse in 9** | **better in 9** | **better 8, worse 0, n.s. 1** |

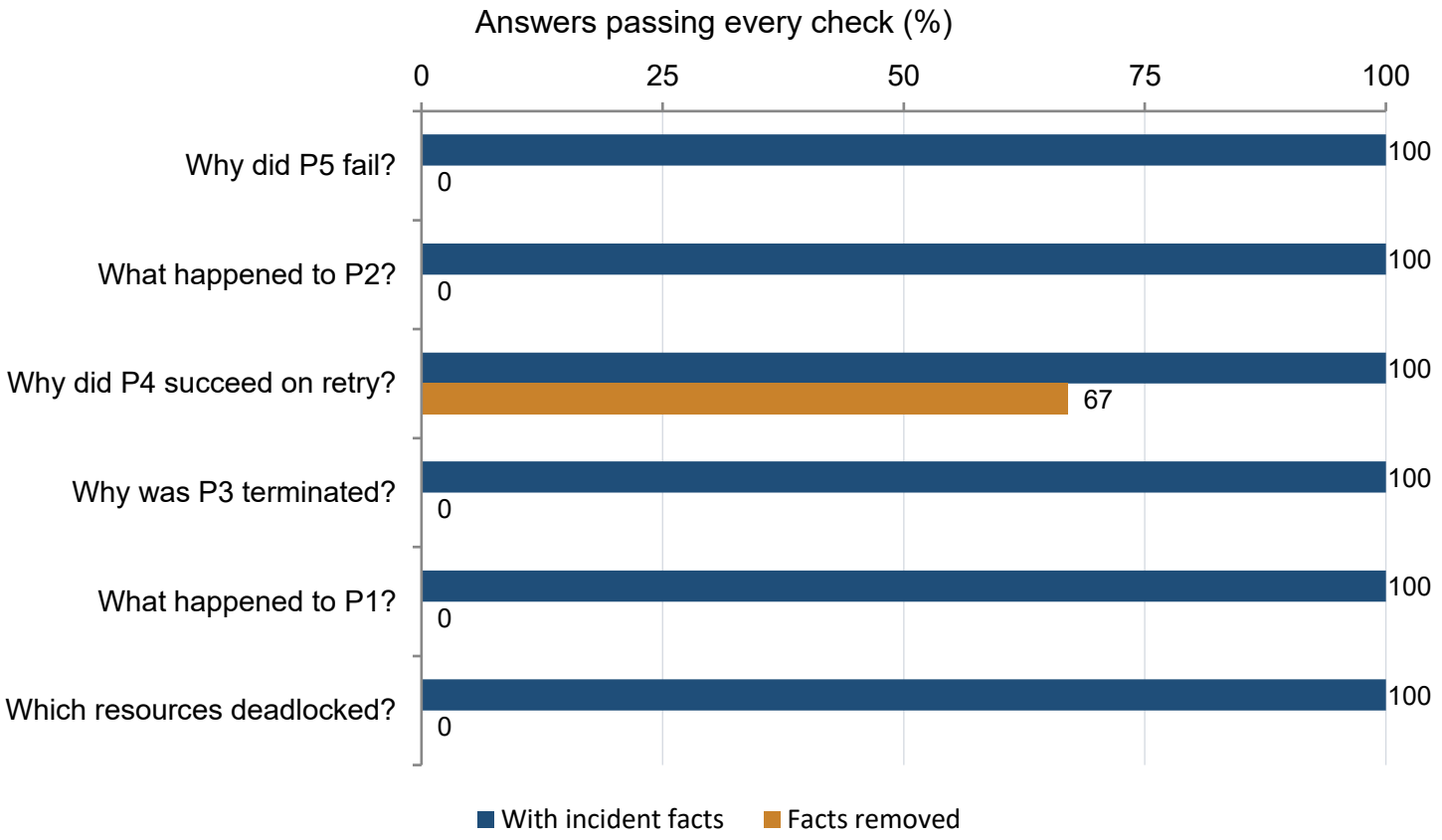
- **SJF** wins on average wait everywhere and loses on worst case everywhere. At load 1.2: 38.5 against 82.7 minutes.
- **Preemption** wins on response time everywhere, but lowers average wait only on heavy-tailed jobs.
- **Priority** as configured is indistinguishable from FCFS.

# Two fairness measures, two different answers

| Measure | What it asks | What it shows |
|---|---|---|
| Jain over CPU received | Did each task get its share of CPU? | Identical for every policy (0.6711 on the seeded run) |
| Jain over 1 / slowdown | Was every task slowed down by a similar factor? | FCFS least fair, SJF fairest (0.71 to 0.78 against 0.45 to 0.60 at 100 jobs) |
| Maximum wait | How long did the unluckiest task wait? | FCFS best; all five others worse in 9 of 9 settings |

The delay-fairness index is my own construction (Jain's index applied to 1/bounded slowdown, bound = 1 minute, fixed before the study). It is documented but it is not a standard metric.

# AI answers scored with and without the recorded facts




| Condition | Pass |
|---|---|
| Incident, with facts | **18 of 18** |
| Incident, facts removed | **2 of 18** |
| Concept questions | 13 of 15 |

- Without the facts the model cannot know P5 asked for 1600 MB or that P3 was the victim.
- Concept misses: Jain's index stated as running from 0 to 1 (it is 1/n to 1).
- Small: 11 questions, 3 trials, one model, pattern-based checks.

# The search tokenizer, measured on held-out questions

| Question set | Tokenizer | Top-1 | Top-3 | MRR |
|---|---|---|---|---|
| Development (36) | original | 77.8% | 97.2% | 0.873 |
| Development (36) | revised | 88.9% | 100% | 0.940 |
| Held-out (20) | original | 90.0% | 100% | 0.950 |
| Held-out (20) | revised | 95.0% | 100% | 0.967 |

- The development set shaped the revision, so its 11-point gain flatters it.
- On the held-out set the gain is one question of 20, within noise. No claim beyond that.
- Two stopwords added after seeing a development miss were removed again (that was tuning to the test).
- The search is still lexical: it cannot match different words for the same idea.

# Fifteen defects found by running the system

- 1 SJF and Priority idled the CPU waiting for late arrivals; the test encoded it
- 2 Gantt chart had about 190 near-empty rows
- 3 AI blamed fragmentation; the log showed 1600 MB requested, 248 MB free
- 4 AI invented a meaning for EEVDF; knowledge base predated it
- 5 Two tasks evicted each other repeatedly
- 6 A task was suspended without freeing enough to help
- 7 Failures handled after memory had already been freed
- 8 Tasks flagged although they already held their memory
- 9 Finished tasks could be flagged and recovered
- 10 Error count stayed 0 after memory failures
- 11 Test suite made live model calls
- 12 App unusable when port 8000 was taken
- 13 Silent offline answers when the model was rate limited
- 14 File handles called megabytes in AI answers
- 15 My evaluation graded correct answers wrong (no-break spaces)

> Two of the fifteen were faults in the measurement itself. All are fixed and, where possible, covered by a regression test.

# What this does not claim

| Limitation | Consequence |
|---|---|
| A simulation on one abstract CPU | It shows policy behaviour, not kernel behaviour |
| Synthetic workloads only; real archive trace not obtained | Rankings are known to depend on the workload; real-trace results are future work |
| Results confirm known theory | The contribution is the measured dependence on workload shape, the delay-fairness index and the grounding evidence |
| AI evaluation is small and pattern-graded | It shows grounding matters; it is not a general quality score |
| Search is lexical TF-IDF | Held-out gain is one question; embeddings could do better |
| Anomaly detector flags abrupt changes only | A steady overrun raises no alarm |
| Suspension releases memory but leaves the task READY | Documented; not a true suspended state |
| No authentication | Single local user; do not expose on a shared network |

# What comes next

- Replay a real Parallel Workloads Archive trace and implement the calibrated Lublin-Feitelson generator.
- Sweep the quantum, boost interval and slice length; extend the simulator to several CPUs.
- A larger question set with human grading; compare embedding retrieval with TF-IDF and keep the offline path.
- Multi-user operation with authentication, so fair-share applies across real users.
- A baseline from completed tasks for the anomaly detector; make suspension a real state.
- Enforce lock ordering instead of only logging violations; add a distributed detector (Chandy-Misra-Haas) as an option.

# Likely questions (1 of 2)

| Question | Answer |
| --- | --- |
| Is this novel? | The algorithms are not: EEVDF, MLFQ, fair-share and the NIST detectors are published. What is new is the combination in one local system, the delay-fairness index, the measured dependence of preemption's benefit on workload shape, and a verification record that includes an ablation of the AI's grounding. |
| Why trust the numbers? | Each table comes from a command in the README with a fixed seed. Comparisons with FCFS are paired with 95% intervals. Fifteen defects were found by running it, fixed and given tests; two were faults in the measurement itself. |
| Why is Round Robin's average wait higher than FCFS? | Time-slicing makes most jobs finish later than running them straight through, and average wait rewards finishing early. It wins only when a few very long jobs block many short ones (heavy-tailed workloads). Its benefit is response time, lower in all nine settings. |

# Likely questions (2 of 2)

| Question | Answer |
| --- | --- |
| If SJF is best on average wait, why not always use it? | It needs burst times in advance, which a user only estimates, and it starves long jobs: its maximum wait is above FCFS's in all nine settings. Procura shows the worst case beside the average. |
| Does the AI choose the recovery action? | No. The model explains in words; fixed scripts act, following Kubernetes-style preemption rules. A wrong explanation cannot cause a wrong action. |
| Where is the lexer and the parser? | There is no compiler. The lexer-like part is tokenize_v2 in rag/ingestion.py; the parser is parse_swf in benchmark/workload.py. Both can be run alone (next slide). |
| What is the point of a deadlock demo in a task manager? | It shows the full detect, diagnose, recover loop on a classic OS problem, with a precise cycle, a fixed resolution rule, and an explanation that cites the recorded facts. |

APPENDIX

# Appendix: run each part on its own

```
# tokenizer (the lexer-like part)
python -c "from rag.ingestion import tokenize; print(tokenize('Why did the Render-video task hit a memory failure on kernel 6.6?'))"
# search
python -c "from rag.retrieval import search; [print(round(r.score,3), r.chunk.source) for r in search('How is EEVDF different from Round Robin?')]"
# SWF parser (the parser-like part)
python -c "from benchmark.workload import parse_swf; j = parse_swf('benchmark/sample_traces/sample.swf'); print(len(j), j[0])"
# schedulers and benchmark
python -m benchmark.run_benchmark --n 30 --arrival-rate 0.3 --seed 42
python -m benchmark.run_benchmark --swf benchmark/sample_traces/sample.swf
python -m benchmark.study                 # 495 workloads; writes benchmark/results/
# evaluations and tests
python -m rag.eval_retrieval              # search accuracy, development and held-out
python -m rag.eval_grounding              # live model; needs a key in backend/.env
python -m pytest tests -q                 # 101 tests
```